

Accountable Agents

Building agent systems as a double-entry provenance ledger

Working draft · v0.1 30 May 2026 A lens, not a circuit

This note proposes one way to build agent systems so that every output is **accountable by construction**: you can always answer “where did this come from, through what, and on whose authority?” without bolting logging on after the fact. The organizing idea is borrowed from accounting – a double-entry ledger, where nothing is recorded without being balanced against its source.

It is deliberately incomplete. It marks its open questions explicitly, and its central claim is that those open questions are not gaps to be closed once and for all – they are the **extension points**, the plugins, where a concrete system supplies its own choices. Read it as a sketch of the schematic, not the finished circuit.

THE TASK, KEPT SMALL

The simplest agent that is still architecturally complete

Picture the smallest useful agent: it receives a request, gathers some facts, does some work – an algorithm, a model call, sometimes a human – checks the result, and commits an answer. We want it built so that (a) every piece of the answer is traceable to its origin; (b) the model’s *generated* contribution is visibly separated from *sourced* facts and *human* decisions; (c) the system can re-run, correct, and audit itself; and (d) a person can step in at exactly the points that matter. What follows is the smallest set of design commitments that gets you there – and a list of the places you will have to decide for yourself.

PART I

One append-only store, many projections

T1 There is one source of truth, and it is an append-only log of entries. Don’t hold the agent’s “state” as mutable objects you overwrite. Hold an immutable log of what happened – every fact ingested, every result produced, every decision taken. Everything else (the current state, any table or graph the code reads) is a view *computed* from the log. This is event sourcing: the log is primary, state is derived. The payoff is that history is not a feature you add – it *is* the system.

T2 The working “graph” is a projection, not a second database. Agents like to think in a graph of entities and relations. Keep that graph as a materialized view over the log – a read model you can always rebuild. There is no consistency problem between “the trace” and “the data,” because they are the same thing read two ways: the log along time, the graph at an instant.

T3 Producers and consumers talk through the store by content, not by name. A step that needs input asks for it by *what it is* – “claims of type X about entity E, confidence over 0.8” – not by *who made it* – “the output of parse_claims.” This is the blackboard / tuple-space pattern: coordination by pattern-matching rather than addressing. It is what makes steps composable – add, remove, or reorder producers without touching consumers.

OPEN · PLUGIN

The query/selector engine – how patterns are matched against the store – is yours to choose, and the attribute vocabulary it matches on is the one thing that cannot be left implicit. See the Plugin Surface.

PART II

The unit of work: a uniform four-phase node

T4 Every step has the same shape, whatever it does. A node has four phases: **resolve** (collect its inputs from the store via selectors), **act** (do the work), **assess** (decide what happens next), **post** (write results back as ledger entries). Uniform shape buys uniform tooling, uniform tracing, and a small mental model. (It is functional-core/imperative-shell: act is the pure-ish core, resolve and post are the I/O shell.)

T5 The act phase is executor-agnostic: code, model, or human, behind one contract. Whether a step runs a deterministic function, calls an LLM, or routes to a person is an internal detail of act. The orchestration does not change when you swap one for another. This is the single most useful property of the whole arrangement: start a capability as a human task, replace it with a model, later replace it with code – without rewiring anything around it.

T6 Cardinality and schema growth live in resolve and post, and they bite. Content-addressed selection returns 0, 1, or N, so every selector needs a cardinality contract (expect-one / expect-many / at-least-one) and a policy when it is violated. And post is not “fill in known fields” – interesting work *creates* structure that was not in the input schema. So post must be allowed to open new structure, and opening it must itself be recorded. Treat schema growth as a logged event, never a silent side effect.

The ledger: double-entry provenance

T7 Every write is a balanced entry: no result may be posted without crediting its sources. This is the spine. Borrow double-entry bookkeeping: you cannot record that something *appeared* without recording what it *came from*. A result debited into the store must be balanced by credits to its inputs, to a model, or to a human. The system *refuses* an unbalanced write – enforced at post time, not checked later in an audit.

T8 The conserved quantity is accountedness of origin – not information. This is the one place people over-read the metaphor, so state it plainly: semantics is *not* conservative. A model call adds content; a consolidation step destroys it. So “information” is not what balances. What balances is *attribution*: every node must have accounted-for provenance. The integrity check is therefore an **orphan check** – the books balance iff no node exists without accounted in-edges.

T9 Tag every contribution by kind: SOURCED, DERIVED, GENERATED, DECIDED. Sourced = from inputs or external data. Derived = transformed by a deterministic step. Generated = produced by a model beyond what the inputs entail. Decided = committed by a human. With these four account types you can read off any answer’s composition – how much was given by the model versus grounded in sources versus owned by a person. In a regulated or high-stakes setting that separation *is* the deliverable. And **generated content is a liability**: it is drawn against a future obligation to validate it, source it, or have a human own it.

T10 Corrections, merges, and failures are entries, never erasures. You never mutate the log. To fix something, post a *reversing entry* (the accountant’s credit memo; in distributed systems, a compensating transaction – a saga). To combine overlapping results, post a *consolidation entry* that credits both and debits the merged result. When a step fails or a restart discards work, you reverse it – the record that it happened survives. Ingesting the original request is itself the *opening entry*: credited from “the outside world,” debited into the store. So even the first decomposition of the request is balanced and traced – there is no privileged, untraceable root.

The control plane: assess returns verbs, not data

T11 The assess phase produces a control decision, not a result. Separate the data plane (what flows) from the control plane (what governs flow). act is data plane; assess is control plane. Its output is one of a small set of verbs – PROCEED, RETRY-FROM-STAGE, ABORT, SKIP-TO-STAGE, ESCALATE-TO-HUMAN – resolved through an explicit table. This is where responsibility lives: not where work is done, but where it is decided whether the work may propagate.

T12 PROCEED is an explicit, logged action – not the absence of one. The commonest verb is “fine, continue.” If continuing is merely what happens when nothing else fires, then the most frequent decision in your system is never recorded, and accountability evaporates exactly where most of the traffic flows. Make PROCEED an affirmative entry – the same reason aviation distinguishes an active decision to continue from a failure to stop.

T13 The verb table needs a fail-closed default, a progress measure, and a privileged-verb discipline. Three safety properties. (a) Any control state not in the table defaults to halt-or-escalate, never proceed. (b) RE'TRY-FROM-STAGE creates a loop, so it must require a measurable reduction in some quantity each pass – unresolved conflicts, ungrounded liabilities – or it livelocks; a restart that reduces nothing is stagnation in the mask of iteration. (c) SKIP-TO-STAGE is the dangerous verb, because it *bypasses* the methodological stages – it must carry an explicit reason and a higher authority than a retry. Changing a threshold in this table is itself a governed, logged act.

PART V

Commitment and human presence

T14 A commit point is where the system commits to an output; commit points have variable height. Not every result needs a human. Cheap, reversible, routine commitments are decided locally by assess. Expensive, irreversible, or novel commitments are handed up to a human locus. The height is per-commitment, not global.

T15 Human presence comes in three modes, and they are not interchangeable. Synchronous (a person decides now), policy (a person authored the mechanical checks in advance), and post-hoc (a person reviews the trace later). These are *before* / *during* / *after* the decision – legislative, executive, judicial. Their risk profiles differ sharply: policy checks are blind to the novel; post-hoc review catches the novel only after the consequence has propagated. (This is the maker-checker / segregation-of-duties principle, and the logic behind named-accountability regimes.)

T16 The required mode is a function of reversibility × novelty. Reversible and anticipated → policy or post-hoc is enough, and you get to scale. Irreversible and novel → you *cannot* delegate to mechanical checks or to later review; a person must be present during the decision. ESCALATE-TO-HUMAN is the mechanism that *promotes* a commitment to synchronous presence when novelty is detected at runtime.

OPEN • PLUGIN • THE HARD ONE

Detecting “this is novel – outside what the checks were authorized for” is an out-of-distribution problem, and it is the most load-bearing component in the whole design. Without it, policy presence silently degrades into *no* presence exactly when novelty arrives.

T17 What a human sees is a projection – and at a decision point it must be full-disclosure. A person never sees the raw store; they see a report scoped to their view-point, with drill-down to the underlying entries. But a report can hide the liabilities – show the polished answer, omit how much of it was GENERATED versus SOURCED. That is the semantic equivalent of off-balance-sheet liabilities. At a decision point the projection must surface the grounding composition, or “human presence” is a signature over hidden liabilities.

Sequencing: stages as discipline, not just schedule

T18 Define an ordered set of stages; their order encodes method, not merely data flow. Beyond the data-dependency DAG (B cannot run before A if B needs A's output), impose a stage order that reflects how you want the work done – establish and verify before you connect, connect before you commit. A stage earns its place only where it forbids an ordering the data dependencies would otherwise permit *and the prohibition matters* (“don’t relate unverified material, because connecting errors propagates them”). If a stage order never constrains anything the dependency graph does not, it is decoration. Stages are a discipline layer.

T19 Data dependencies always win over stage order, and the pipeline must be able to loop. When methodological order and causal order conflict, causal wins – you cannot consume what has not been produced. And a pure forward pipeline can only proceed; to revise, it needs a back-edge (the RETRY-FROM-STAGE verb) governed by the progress measure of T13. That back-edge, gated by a real evaluative pause rather than a reflex, is what separates a system that *iterates* from one that merely *proliferates*.

WHERE THE METAPHOR STRAINS

Read this before you over-trust the ledger

L1 You get integrity, not arithmetic. The balance is *topological* (every node has accounted in-edges), not *numeric*. You cannot honestly say “30% of this output came from input A” – clean attribution in generative systems is the unsolved interpretability problem. Build the orphan check; do *not* build a layer that prints attribution percentages, because it would be fiction wearing accounting’s authority.

L2 Provenance is fixed; meaning drifts. A result’s *origin* is fixed by its entry, but its *significance* is relational – it shifts as the graph grows around it. “Always traceable” is true of where a result came from; it is not true of what it currently means.

L3 Consolidation is a write-off, not a transfer. A step that collapses many results into one *destroys detail on purpose*. Accounting has no “loss of information,” so model it as a write-off: a consolidation must record what it dropped and why, or the trace goes blind at the most consequential operation.

L4 Keep two ledgers, not one. The semantic ledger (accountedness of meaning) and the resource ledger (compute, model cost, human-minutes) are both double-entry but in different currencies. Merging them is a category error; the budget that governs escalation lives in the second ledger.

The open questions are the extension points

The following are deliberately *not* fixed here. Each is an interface a concrete system supplies. The claim of this note is that these are not loose ends but the *seams* – the places the architecture is designed to be specialized, and the places where most of the real engineering and most of the domain judgment actually live.

Attribute ontology	The vocabulary resolve / post match on. Cannot be implicit: when producers and consumers drift in naming, selectors silently under-match. The one mandatory, policed contract.
Selector engine	How patterns are matched against the store, and how cardinality violations are handled.
Ingress decomposer	Turns a raw request into opening entries. Must itself be a logged node, not a privileged root.
Stage set & order	Your domain's discipline layer (T18) – and the prohibitions that make it more than decoration.
Assess → verb table	The verbs, the thresholds, the fail-closed default. Encodes the system's risk appetite.
Merge / write-off policy	When overlapping results coexist versus combine; how conflict is resolved; what a consolidation records as dropped (L3).
Novelty detector	Drives ESCALATE; an out-of-distribution problem; the hardest component (T16).
Attribution model	How the SOURCED / DERIVED / GENERATED / DECIDED split is assigned – bounded by L1: topological, not numeric.
Authority model	Who may post which entry types; who may move a threshold; segregation of duties.
Presence routing	The reversibility × novelty → presence-mode function (T16).

Resource ledger	The second ledger; budgets and credit limits that force settlement or escalation (L4).
Projection specs	Per view-point reports, with the full-disclosure requirement at decision points (T17).

REFERENCES

Prior art this leans on

Blackboard architecture – Hayes-Roth; the HEARSAY-II speech-understanding system. Shared structured memory plus opportunistic knowledge sources; the hard part was always the controller.

Tuple spaces / Linda – Gelernter & Carriero, 1985. Coordination by pattern-matching, not by addressing.

Event sourcing & CQRS – Fowler; Greg Young. Append-only log as truth; state as projection.

Double-entry bookkeeping – Pacioli, 1494. Balanced entries, conservation, an audit trail by construction.

Functional core, imperative shell – Gary Bernhardt. Pure transformation, effects at the boundary.

Sagas / compensating transactions – Garcia-Molina & Salem, 1987. Reversal instead of rollback.

Segregation of duties / maker-checker – accounting and security controls; authorization on actions.

DAG orchestration – Airflow, Dagster, Temporal; LangGraph for agent control flow.

Out-of-distribution / novelty detection – the runtime trigger for escalation.

Data provenance / lineage – W3C PROV; the derivation-edge model.

Retrieval-augmented generation / grounding – the SOURCED-versus-GENERATED line.

CLOSING

A narrow wager, stated plainly

None of the parts here are new on their own; the observation is only that they compose. The wager is narrow and worth stating without inflation: **if you make accountedness the conserved quantity and enforce it at write time, you get provenance, correction, and audit almost for free – and you make a system’s honesty about what it generated mechanical rather than aspirational.**

That is a step on one axis only. It does not make the underlying semantics easier; it makes the bookkeeping over that difficulty trustworthy. Everything genuinely hard – novelty detection, attribution, meaning under drift – has been named, not solved, and located where it belongs: in the plugins.

APPENDIX · ORIGIN (OPTIONAL)

The plain terms above generalize a more specific authored grammar. A compact mapping, for continuity:

store / log ↔ **indestructible trace**
balanced entry ↔ **double-entry over a chart of accounts**
generated-as-liability ↔ **informational overdraft / fold-debt**
consolidation ↔ **fold** · decomposition ↔ **unfold**
assess-verbs ↔ **govern / stop**
stages ↔ **Harmogics phases**
commit point ↔ **the committing fold / Responsibility**